Programming and Operating Systems

KH5039CMD

Lab 5.2 – System Fork

School of Computing

The Knowledge Hub Universities

partnered with Coventry University

Process

- **Process:** In an operating system, a process is an instance of a program that is currently running. It is a separate entity with its own memory, resources, CPU, I/O hardware, and files.

- **Parent Process:** The process that uses the fork system call to start a new child process is referred to as the parent process. It acts as the parent process's beginning point and can go on running after the fork.

- **Child Process:** The newly generated process as a consequence of the Fork system call is referred to as the child process. It has its own distinct process ID (PID), and memory, and is a duplicate of the parent process.

- **Process ID:** A process ID (PID) is a special identification that the operating system assigns to each process.

- **Return Value:** The fork system call's return value gives both the parent and child process information. It helps in handling mistakes during process formation and determining the execution route. (0,1,-1)

Task 1 – Creating a **process**

1. Open your VM and start VS Code.
2. Create a folder called “Lab5”
3. Create a new subfolder that is called “Task1”
4. Open this subfolder folder on VS Code and create a C file that is called “**Task1.c**”
5. Write a Hello World C program. This program should print “ **Hello World** “ and the **PID** of this running process.

To get the PID of a running process use the function **getpid()**.
getpid() : Returns the process ID of the calling process.

Task 1 – Creating a process

```
#include <stdio.h>
#include <unistd.h>

int main(){
printf("Hello World, my pid is %d\n", getpid());

return 0;   }
```

```
Hello World , my pid is 2587
```


The Knowledge Hub Universities
partnered with Coventry University

Task 2 – Running a Process from the terminal

1. Create a new subfolder in “Lab5” folder that is called “Task2”.

2. Now Open this subfolder on VS Code and create a C file that is called “Task2.c”.

3. Write a C program that takes your first name as an input and output your name. Save your file.

Task 2 – Running a Process from the terminal

```
#include <stdio.h>
int main () {
char name[20] ;
printf ("Please enter your first name: " ) ;
scanf ("%s", name) ;
printf ("Welcome %s \n", name) ;
}
```

Task 2 – Running a Process from the terminal

4. Copy the path of your "Task2" subfolder. You can do so by opening the subfolder and then press "Ctrl+L" , the default path bar becomes a location entry and then you can copy the path.

5. Open a Terminal and type :

cd "Folder Location"

You can simply paste the subfolder location in the terminal. ***cd*** will simply change the current working directory in Ubuntu to the ***"Folder Location"***.

Task 2 – Running a Process from the terminal

6. Type :

gcc Task2.c –o Task2

This invokes the compiler which transforms your program into the assembly and machine code that your computer truly understands. When the command is executed, it will then create a file called “Task2” that you can run to execute your program.

7. Now run your program by typing the following command :

./Task2

Task 2 – Running a Process from the terminal

```
ahmed@ahmed-VirtualBox:~$ cd /home/ahmed/lab2/Task2
ahmed@ahmed-VirtualBox:~/lab2/Task2$ gcc Task2.c -o Task2
ahmed@ahmed-VirtualBox:~/lab2/Task2$ ./Task2
Please enter your name: Ahmed
Welcome Ahmed
ahmed@ahmed-VirtualBox:~/lab2/Task2$
```

- Process system calls :
  - fork()
  - exec()
  - wait()

Task 3

- Write and execute the code in the next slide and try to explain the output.

Fork() system call

- Below are different values returned by **fork().**

- ***Negative Value***: The creation of a child process was unsuccessful.

- ***Zero***: Returned to the newly created child process.

- ***Positive value***: Returned to parent or caller. The value contains the process ID of the newly created child process.

Task 4

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{

    // make two process which run same
    // program after this instruction
    pid_t p = fork();
    if(p<0){
      perror("fork fail");
      exit(1);
    }
    printf("Hello world!, process_id(pid) = %d \n",getpid());
    return 0;
}
```

```
Hello world!, process_id(pid) = 31
Hello world!, process_id(pid) = 32
```

Task 5

```
Hi,I am the parent with pid 3911
 Fork returned : 3912
 I am the process with pid 3911
 Fork returned : 0
 I am the process with pid 3912
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main (){
printf("Hi,I am the parent with pid %d \n ",getpid());
int rc = fork();
printf("Fork returned : %d \n ",rc);
printf("I am the process with pid %d \n ",getpid());
wait(NULL);
return 0;
}
```

Task 5

- In the first print statement , we have printed the PID of the parent process using getpid().

- After that , the parent has forked a child using the fork() system call , where the child will have the exact same copy of code that is after the call to fork() , with one difference , which is the value of rc returned by the fork() system call. For the child ,rc = 0 and for the parent rc = child's pid .
- Next , the parent has executed its code, where it has printed the return value of fork() (its copy of rc (return code) which is simply the pid of the child) and then it has printed it's pid using the last print statement in its code. Finally , it has waited for the child to end its execution using the wait() system call.

Task 5

- The remaining print statements in the output are that of the child , where it has executed the exact same code as that of the parent .
- Remember, rc is equal to 0 for the child , and hence its first print statement was “Fork returned : 0” .
- Finally , the child has printed out the last print statement which is it’s pid that is the same as the rc value returned to the parent (see the second print statement in the output).

Task 5

Parent

```
printf("Fork returned : %d \n ",rc);
printf("I am the process with pid %d \n ",getpid());
wait(NULL);
return 0;
```

Child

```
printf("Fork returned : %d \n ",rc);
printf("I am the process with pid %d \n ",getpid());
wait(NULL);
return 0;
```

```
Hi,I am the parent with pid 3911
 Fork returned : 3912
 I am the process with pid 3911
 Fork returned : 0
 I am the process with pid 3912
  ahmed@ahmed-VirtualBox:~/lab5$
```

Fork() in greater details

- Fork() is used to create new process.
- On its return, fork () creates a *(child)* process that is an (almost) exact copy of the calling *(parent)* process.
- The child process does not execute from the beginning of main, but rather it starts executing the code just after the fork () call.
- The child process will have its own
  - Address space in memory
  - Registers
  - PC
  - Etc…
- The value returned (returned code rc) from fork in the child differs from that returned in the parent
  - rc at the child = 0
  - rc at the parent = child PID
- On failure, -1 is returned as the rc in the parent, no child process is created in this case.

Task 6

- Create a parent process that forks a child process. Your code should print the parent and child PIDs separately. Also, your code should print “Error creating a new process” , if no child process is created due to an error while forking.
- Sample output :

```
Hello World, my pid is 4203
Hello, I am the parent, my pid = 4203
Hello, I am the child, my pid = 4204
ahmed@ahmed-VirtualBox:~/lab5$
```

Task 6- Solution

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
printf("Hello World, my pid is %d\n", getpid());
int rc = fork();
if(rc<0){
 printf("Error creating a new processs\n");
exit(1);}
if(rc == 0) //child process
{
printf("Hello, I am the child, my pid = %d\n",getpid());
}
else //Parent process
{
printf("Hello, I am the parent, my pid = %d\n",getpid());
wait(NULL);
}
return 0;   }
```

Task 7

- Remove the function call to wait() in your code of Taks2. Can you explain the output ?

```
Hello World, my pid is 4265
Hello, I am the parent, my pid = 4265
ahmed@ahmed-VirtualBox:~/lab5$ Hello, I am the child, my pid = 4266
```

Task 7- Output Explanation

- Simply , the parent executes its code section and ends before the termination of the child process. Since the child didn't not finish its execution , after the shell issues its prompt the child prints its output (and then terminates).
- Note that there is no guarantee of order between processes. The OS ***schedule***r can order them any way it wants , so that the child could have printed its output before the parent.
- So, to impose that a parent finishes after the child, use the wait () system call.

The Knowledge Hub Universities
partnered with Coventry University

wait() system call

- wait () will force the parent to wait for the child to finish.
  - ✓ https://man7.org/linux/man-pages/man2/wait.2.html
- If a child terminates, performing a wait allows the system to release the resources associated with the child; if a wait is not performed, then the terminated child remains in a "zombie" state.
- A zombie process is a process whose execution is completed but it still has an entry in the process table.
- Once the parent calls wait(), the process identifier of the zombie process and its entry in the process table are released.
- if a parent did not invoke wait() and instead terminated, thereby leaving its child processes as orphans.

Task 7: multiple forks()

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    fork();
    fork();
    fork();
    printf("hello\n");
    return 0;
}
```

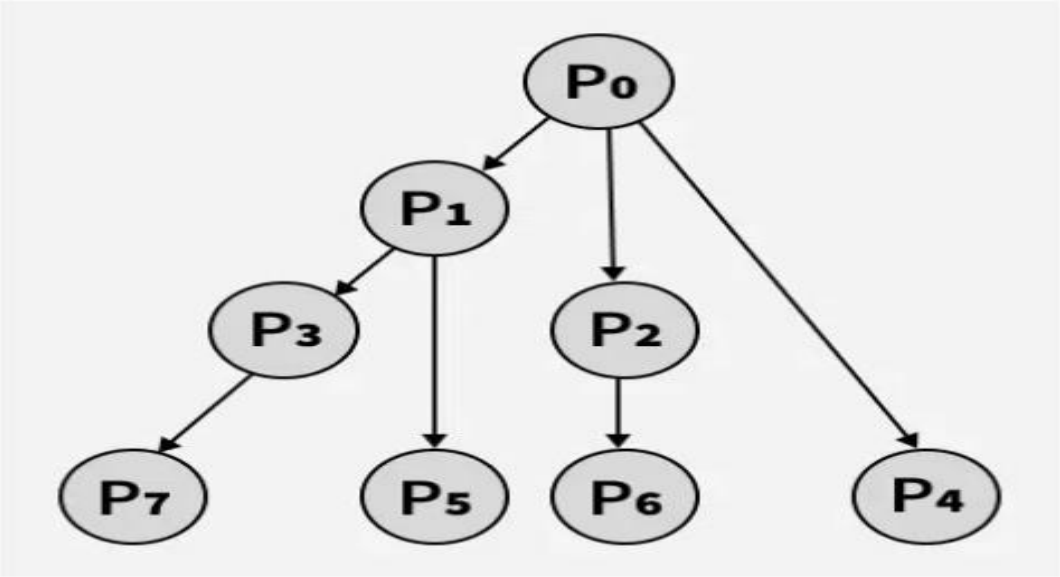
```
hello
hello
hello
hello
hello
hello
hello
hello
```

Task 7: multiple forks ()

```
fork ();   // Line 1
fork ();   // Line 2
fork ();   // Line 3
       L1        // There will be 1 child process
    /     \      // created by line 1.
  L2      L2     // There will be 2 child processes
 /  \    /  \    //  created by line 2
L3  L3  L3  L3   // There will be 4 child processes
                 // created by line 3
```

Task 7: multiple forks

- Main process: **P0**
- Processes created by the **1st fork: P1**
- Processes created by the **2nd fork: P2, P3**
- Processes created by the **3rd fork: P4, P5, P6, P7**



```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
void forkexample()
{
    pid_t p;
    p = fork();
    if(p<0)
    {
      perror("fork fail");
      exit(1);
    }
    // child process because return value zero
    else if ( p == 0)
        printf("Hello from Child!\n");

    // parent process because return value non-zero.
    else
        printf("Hello from Parent!\n");
}
int main()
{
    forkexample();
    return 0;
}
```

Task 8

```
Hello from Parent!
Hello from Child!
```

Task 8

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

void forkexample()
{
    int x = 1;
    pid_t p = fork();
      if(p<0){
      perror("fork fail");
      exit(1);
    }
    else if (p == 0)
        printf("Child has x = %d\n", ++x);
    else
        printf("Parent has x = %d\n", --x);
}
int main()
{
    forkexample();
    return 0;
}
```

```
Parent has x = 0
Child has x = 2
```


The Knowledge Hub Universities
partnered with Coventry University

Thank you